

SmartCow Tracker

Sistema de Rastreo y Monitoreo de Ganado Bovino

INTERFACE DESIGN DOCUMENT (IDD)

IEEE Std 1016-2009 · OpenAPI 3.1 · W3C WCAG 2.1 · Material Design 3

Versión	1.0.0
Fecha	Mayo 2026
Basado en	IEEE 1016-2009 · OpenAPI 3.1
Estado	Borrador — Revisión inicial

Historial de Revisiones

Versión	Fecha	Descripción	Autor
1.0.0	28/05/2026	Versión inicial IDD — SmartCow Tracker	Equipo SmartCow

1. Introducción

1.1 Propósito

El presente Interface Design Document (IDD) especifica el diseño detallado de todas las interfaces del sistema SmartCow Tracker: la interfaz de usuario web (UI), la interfaz de programación de aplicaciones REST (API), los contratos de mensajería con dispositivos IoT via MQTT, los eventos WebSocket de tiempo real y los esquemas de datos de integración con servicios externos. Este documento se elabora conforme al estándar IEEE Std 1016-2009 y complementa el SDD (diseño interno) con el diseño de las fronteras del sistema.

Su propósito es servir como contrato técnico entre los equipos de frontend, backend, hardware IoT y servicios externos, garantizando que todas las interfaces sean consistentes, versionadas y documentadas de forma verificable.

1.2 Alcance

Este IDD cubre las siguientes interfaces del sistema:

- Interfaz de usuario web (UI): sistema de diseño, tokens de diseño, componentes reutilizables, layouts de pantallas principales y flujos de navegación.
- Interfaz REST API: contratos OpenAPI 3.1 de todos los endpoints con esquemas de request/response, códigos de estado y validaciones.
- Interfaz MQTT IoT: estructura de topics, esquemas de payloads y protocolo de autenticación de dispositivos.
- Interfaz WebSocket: eventos de tiempo real, estructura de namespaces y ciclo de vida de conexión.
- Interfaces de servicios externos: contratos de integración con SendGrid, Twilio, Mapbox/OSM y S3.

1.3 Referencias

- IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.
- OpenAPI Specification 3.1.0 — <https://spec.openapis.org/oas/v3.1.0>
- W3C WCAG 2.1 — Web Content Accessibility Guidelines.
- Material Design 3 — Sistema de diseño de Google (<https://m3.material.io>).
- RFC 6455 — The WebSocket Protocol.
- MQTT 3.1.1 Specification — OASIS Standard.
- SmartCow Tracker SDD v1.0.0 — Software Design Document.

2. Sistema de Diseño de la Interfaz de Usuario

2.1 Principios de Diseño

La interfaz de SmartCow Tracker sigue cuatro principios fundamentales que guían todas las decisiones de diseño visual e interacción:

Principio	Descripción y aplicación
Claridad operacional	Un ganadero en campo debe poder identificar el estado de su hato en menos de 5 segundos. La información crítica (alertas, animales fuera de zona) siempre tiene prioridad visual sobre la información contextual.
Modo oscuro primero	El modo oscuro es el predeterminado. Los ganaderos usan tablets y monitores en condiciones de luz variable (al aire libre, en galpones). El modo oscuro reduce la fatiga visual y el consumo de batería en pantallas OLED.
Semántica de color	Verde = normal/seguro. Ámbar = advertencia/monitorear. Rojo = crítico/acción inmediata. Gris = sin datos/offline. Esta semántica se aplica de forma consistente en marcadores de mapa, badges, alertas y gráficos.
Accesibilidad WCAG 2.1 AA	Contraste mínimo 4.5:1 para texto normal y 3:1 para texto grande. Navegación por teclado completa. Etiquetas ARIA en todos los elementos interactivos. Tamaño mínimo de objetivo táctil: 44×44px.

2.2 Tokens de Diseño — Paleta de Colores

Los tokens de diseño son las variables CSS que definen el sistema visual. Se organizan en dos temas: oscuro (predeterminado) y claro (opcional).

2.2.1 Colores Semánticos del Dominio

Token CSS	Modo Oscuro	Modo Claro	Uso
--color-primary	#1a7a4a	#1a7a4a	Botones primarios, acentos, iconos de acción principal.
--color-primary-light	#22a05c	#1a7a4a	Hover de botones, estados activos, indicadores online.
--color-status-ok	#4caf50	#2e7d32	Animal sano, sensor online, geocerca respetada.
--color-status-warning	#ffb300	#e65100	Alerta de advertencia, temperatura elevada, batería baja.
--color-status-critical	#f44336	#b71c1c	Alerta crítica, animal fuera de zona, posible robo.

--color-status-offline	#6e7681	#9e9e9e	Sensor offline, animal sin datos recientes.
--bg-surface-primary	#0d1117	#ffffff	Fondo principal de la aplicación.
--bg-surface-card	#161b22	#f5f5f5	Tarjetas, paneles, modales.
--bg-surface-elevated	#21262d	#eeeeee	Elementos elevados: dropdown, tooltips, popovers.
--text-primary	#e6edf3	#212121	Texto principal de alta legibilidad.
--text-secondary	#8b949e	#616161	Texto secundario, etiquetas, metadatos.
--border-default	#30363d	#e0e0e0	Bordes de tarjetas, divisores, inputs.

2.3 Tokens de Diseño — Tipografía

Elemento	Fuente	Tamaño	Peso	Uso
--font-display	Inter	32-48px	700	Títulos de página y métricas grandes del dashboard.
--font-heading-xl	Inter	24px	600	Encabezados de sección principales (H1 de pantalla).
--font-heading-lg	Inter	20px	600	Títulos de tarjetas y módulos (H2).
--font-heading-md	Inter	16px	500	Subtítulos, nombres de animales, encabezados de tabla.
--font-body	Inter	14px	400	Cuerpo de texto, descripciones, contenido general.
--font-small	Inter	12px	400	Metadatos, timestamps, etiquetas de campos.
--font-mono	JetBrains Mono	13px	400	Coordenadas GPS, IDs de sensores, códigos técnicos.
--font-badge	Inter	11px	600	Badges de estado, etiquetas de alerta, chips de filtro.

2.4 Tokens de Diseño — Espaciado y Geometría

Token	Valor	Uso principal	
--space-1	4px	Espaciado mínimo entre elementos inline.	
--space-2	8px	Padding interno de badges, separación de iconos y texto.	
--space-3	12px	Gap entre elementos de una fila, padding de chips.	
--space-4	16px	Padding estándar de tarjetas y paneles.	
--space-6	24px	Gap entre tarjetas en grillas, separación de secciones.	
--space-8	32px	Margen de módulos, separación de grupos principales.	
--radius-sm	6px	Badges, chips, botones pequeños.	
--radius-md	10px	Inputs, botones estándar, tooltips.	
--radius-lg	14px	Tarjetas de ganado, paneles de mapa, modales.	
--radius-xl	20px	Bottom sheets, drawer lateral, cards hero.	
--shadow-card	0 1px 3px rgba(0,0,0,0.4)	Elevación de tarjetas en modo oscuro.	
--transition-fast	150ms ease-out	Hover de botones, cambio de estado de badges.	
--transition-normal	250ms ease-in-out	Apertura de modales, expansión de paneles.	

3. Componentes de Interfaz Reutilizables

Esta sección especifica los componentes UI reutilizables del sistema de diseño. Cada componente define sus variantes, estados, propiedades y comportamiento de accesibilidad.

3.1 Componente: StatusBadge

Indicador visual compacto del estado de un animal, sensor o alerta. Aparece en listados, tarjetas de ganado y en el panel de detalle.

Variante	Color fondo	Color texto	Cuándo usar
healthy	#e8f5e9	#1b5e20	Animal con signos vitales normales y dentro de geocerca.
warning	#fff8e1	#e65100	Temperatura elevada, batería < 30%, inactividad moderada.
critical	#ffebee	#b71c1c	Animal fuera de geocerca, temperatura crítica, posible robo.
offline	#f5f5f5	#424242	Sensor sin señal por más de 5 minutos.
pregnant	#f3e5f5	#6a1b9a	Animal en estado de gestación registrado.

Accesibilidad: No depender solo del color. Cada badge incluye un ícono SVG a la izquierda del texto para usuarios con daltonismo.

3.2 Componente: AnimalCard

Tarjeta de resumen de un animal. Se usa en el listado de ganado y en el panel de acceso rápido del dashboard.

Propiedad	Especificación
Dimensiones	Ancho: 100% del contenedor. Alto: 80px (compacto) / 110px (expandido con vitales). Border-radius: var(--radius-lg).
Estructura	Columna izquierda: emoji del animal + avatar de color por estado (40×40px). Centro: nombre (font-heading-md), metadata (raza, código, font-small, text-secondary). Derecha: vitales actuales (temperatura, bpm) + StatusBadge.
Estado hover	Border: 1px solid var(--color-primary). Fondo: bg-surface-elevated. Transición: 150ms. Cursor: pointer.
Estado seleccionado	Border: 2px solid var(--color-primary). Fondo: rgba(26,122,74,0.12). El panel de detalle lateral se abre.
Interacción	Click → abre CattleDetailPanel. Click en ícono de mapa → centra el mapa en ese animal. El estado (activo/expandido) se almacena en Redux.

Accesibilidad	role='button', aria-label='Ver ficha de {nombre}', tabIndex=0. Activable con Enter y Space.
---------------	---

3.3 Componente: VitalCard

Tarjeta individual de signo vital con valor actual, tendencia y mini-gráfico sparkline. Se agrupa en conjuntos de 4 en la ficha del animal.

Elemento	Especificación
Header	Ícono del signo vital (16px, aria-hidden) + etiqueta en font-small + text-secondary. Ej: '  Temperatura'.
Valor principal	font-display (28px, 700). Color semántico: text-primary si normal, --color-status-warn si alerta, --color-status-critical si crítico. Unidad en font-body junto al valor.
Indicador de tendencia	Flecha ↑↓↔ + texto descriptivo en font-small. Colores: verde (normal), ámbar (subiendo hacia umbral), rojo (encima del umbral).
Sparkline	Mini-gráfico SVG de las últimas 5 lecturas. Altura: 28px. Color de línea: color semántico del estado actual. Se actualiza en tiempo real via Redux.
Animación	Cuando el valor cambia: flash de fondo (0.3s) con el color semántico correspondiente. Si pasa a estado crítico: pulso de borde rojo (1.5s loop).

3.4 Componente: AlertItem

Elemento de lista para una alerta individual. Aparece en el feed de alertas del dashboard y en el AlertCenter.

Elemento	Especificación
Borde lateral	Border-left: 4px solid. Color: --color-status-critical (rojo) / --color-status-warn (ámbar) / --color-status-ok (verde). Siempre visible como indicador rápido de prioridad.
Ícono de tipo	Ícono Tabler de 20px en contenedor de 36×36px con fondo semitransparente del color de prioridad. Ejemplos: ti-map-pin-off (geocerca), ti-thermometer (temperatura), ti-zzz (inmovilidad).
Título	font-heading-md (16px, 500). Texto descriptivo de la alerta. Ej: 'Res #B-042 fuera de geocerca norte'.
Descripción	font-body (14px, 400). Detalle del evento con coordenadas o valores. text-secondary.
Timestamp	font-small (12px). Tiempo relativo ('hace 2 min') que se actualiza en vivo. Al hacer hover, tooltip con timestamp absoluto ISO 8601.
Acciones rápidas	Botones ghost: 'Ver en mapa', 'Ver res', 'Reconocer'. Visible on hover (desktop) / siempre visible (mobile). Tamaño mínimo: 44px de altura.

3.5 Componente: MapMarker

Marcador de animal en el mapa interactivo GPS. Debe ser legible a diferentes niveles de zoom y distinguible por estado de salud.

Estado	Apariencia	Especificación técnica
Normal	Círculo verde sólido	Div: 14×14px, border-radius:50%, background:#4caf50, border:2px solid #fff. Anillo pulsante: 28×28px, opacity 0.3, animación pulse 2s infinite.
Alerta	Círculo ámbar pulsante	Div: 14×14px, background:#ffb300, border:2px solid #fff. Anillo pulsante ámbar. Frecuencia de pulso: 1.5s (más urgente que normal).
Crítico	Círculo rojo pulsante rápido	Div: 16×16px (más grande), background:#f44336. Anillo rojo. Pulso: 1s (mayor urgencia visual). Z-index elevado para quedar sobre otros marcadores.
Offline	Círculo gris estático	Div: 12×12px, background:#6e7681, opacity:0.7. Sin anillo pulsante (inactivo).
Popup (click)	Tarjeta flotante	bg-surface-card, border-radius:var(--radius-lg), padding:12px, box-shadow. Contenido: nombre, estado, temperatura, ritmo cardíaco, última actualización. Botón 'Ver ficha completa'.

4. Layouts de Pantallas Principales

4.1 Layout Base — Shell de la Aplicación

Todas las pantallas de la aplicación comparten el mismo shell (estructura exterior). El shell es responsivo con dos modos:

Zona	Dimensiones (Desktop)	Comportamiento
Sidebar de navegación	Ancho: 240px (expandido) / 64px (colapsado). Fijo en el lado izquierdo.	Colapsa a iconos al hacer click en el botón de toggle. Estado persistido en localStorage. En mobile (<768px) se convierte en bottom navigation bar.
Header superior	Alto: 56px. Sticky (siempre visible al hacer scroll).	Contiene: logo, nombre de la finca activa, buscador global, campana de alertas con badge, avatar del usuario.
Área de contenido	Ocupa el espacio restante. Padding: 24px.	Scroll vertical. En pantallas con mapa (MapView): no-overflow, el mapa ocupa el 100% del área disponible.
Panel lateral derecho	Ancho: 360px. Deslizante desde la derecha.	Se activa al hacer click en un animal (detalle), una alerta (contexto) o una geocerca (edición). Slide-in animation 250ms ease-in-out.

4.2 Dashboard Principal

Pantalla de inicio que muestra el resumen operacional de la finca. Diseñada para responder la pregunta: '¿Todo está bien con mi hato ahora mismo?'

Zona del dashboard	Componentes	Datos y comportamiento
Fila de métricas (top)	4 × MetricCard en grid 4 columnas.	Total ganado / En zona segura / Alertas activas / Sensores OK. Cada card tiene icono, valor grande y sub-texto con color semántico. Se actualiza cada 30s.
Mapa miniatura	MiniMapCard con Leaflet embedded.	Altura: 200px. Muestra todos los marcadores. No interactivo (solo lectura). Botón 'Ver mapa completo' en la esquina inferior derecha.
Feed de alertas	Lista de 3 AlertItem más recientes.	Solo alertas OPEN. Ordenadas por prioridad descendente. Link 'Ver todas' al AlertCenter. Se actualiza en tiempo real via WebSocket.

Acceso rápido	CattleChip horizontal scrollable.	Los 8 animales más recientes o con alertas activas. Dot de color por estado. Click navega a ficha del animal.
Gráfico de actividad	LineChart (Recharts).	Actividad promedio del hato en las últimas 24h. Eje X: horas. Eje Y: índice 0-100. Tooltip al hover con valor exacto.

4.3 MapView — Mapa GPS en Tiempo Real

Pantalla de mapa completo. El mapa ocupa el 100% del área de contenido disponible (viewport completo menos el sidebar).

Elemento	Especificación
Barra de búsqueda flotante	Posición: top:12px, left:12px, right:72px. z-index:1000. Input con debounce 300ms que filtra marcadores visibles en el mapa. Ícono de búsqueda + botón de filtros.
Controles de mapa	Posición: top:12px, right:12px. Botones de zoom (+/-), centrar en finca (ícono home), selector de capas (satélite/calles/híbrido), herramienta de geocercas.
Capa de geocercas	Polígonos con border: 2px dashed, color configurado por geocerca (default: --color-primary). Relleno: rgba del color al 8% de opacidad. Etiqueta del nombre flotante en el centroide.
Panel inferior de estado	Altura: 72px. Sticky bottom. Fondo: bg-surface-card con blur backdrop. Muestra: total animales activos, indicador GPS live (dot animado + texto), leyenda de colores.
Modo dibujo de geocerca	Al activar: cursor cambia a crosshair, overlay semitransparente en el mapa. Los clicks del usuario crean vértices del polígono. Mínimo 3 vértices. Botón 'Completar' cierra el polígono y abre el formulario de nombre.
Rutas de historial	Al activar en panel lateral: polilínea sobre el mapa con gradiente de color (azul=inicio → verde=fin). Puntos de parada marcados. Selector de rango de fechas con picker.

4.4 CattleDetail — Ficha del Animal

Panel lateral deslizante (360px) que muestra la ficha completa de un animal. Se activa desde el mapa (click en marcador) o desde el listado.

Sección	Contenido y comportamiento
Header del panel	Botón de cierre (X) top-right. Emoji del animal + nombre (font-heading-xl) + código. StatusBadge de estado general. Botón '...' con menú: Editar, Asignar alerta, Exportar historial.

Indicador GPS live	Barra de estado con dot animado verde + texto 'En vivo · Sector X'. Coordenadas en font-mono a la derecha. Si offline: dot gris + 'Última señal: hace X min'.
Grid de vitales	4 × VitalCard en grid 2×2. Temperatura, Ritmo cardíaco, Actividad, Batería del sensor. Cada card se actualiza en tiempo real via WebSocket.
Ficha técnica	Lista de pares key-value: Raza, Peso, Edad, Última vacuna, Estado reproductivo, Inmovilidad hoy, Predicción IA. Separados por finos divisores.
Barra de acciones	3 botones de ancho completo: 'Localizar en mapa' (primary), 'Ver historial' (secondary), 'Registrar evento de salud' (secondary). Plus: botón rojo 'Alerta de emergencia'.
Predicción IA	Card colapsable con riesgo en escala de colores (LOW/MEDIUM/HIGH), barra de progreso del score, top-3 features de la predicción y recomendación textual.

5. Contratos de la API REST

5.1 Especificación OpenAPI 3.1 — Información General

Campo OpenAPI	Valor
openapi	3.1.0
info.title	SmartCow Tracker API
info.version	1.0.0
servers[0].url	https://api.smartcow.app/api/v1
servers[1].url	https://staging-api.smartcow.app/api/v1
servers[2].url	http://localhost:4000/api/v1
securitySchemes	BearerAuth: type http, scheme bearer, bearerFormat JWT. Aplicado globalmente a todos los endpoints excepto /auth/login.
components.schemas	Animal, Geofence, Alert, User, GPSPoint, VitalRecord, PaginatedResult<T>, ErrorResponse.

5.2 Esquemas de Datos Principales (JSON Schema)

5.2.1 Esquema: Animal

Campo	Tipo	Req.	Descripción y validaciones
id	string (cuid)	✗ No	Generado por el servidor. No se incluye en POST.
farmId	string (cuid)	✓ Sí	ID de la finca propietaria. Extraído del JWT, no enviado por el cliente.
code	string	✓ Sí	Código único del animal en la finca. 1-20 caracteres alfanuméricos. Pattern: <code>^[A-Z0-9-]+</code> \$
name	string null	✗ No	Nombre del animal. Máximo 50 caracteres.
breed	string	✓ Sí	Raza del animal. Enum: BRAHMAN HOLSTEIN ANGUS SIMMENTAL CEBUINO OTRO
sex	string	✓ Sí	Sexo. Enum: MALE FEMALE
birthDate	string (ISO 8601)	✗ No	Fecha de nacimiento. Formato: YYYY-MM-DD. No puede ser fecha futura.
weightKg	number	✗ No	Peso en kilogramos. Min: 10, Max: 2000.
photoUrl	string (URI)	✗ No	URL de foto almacenada en S3. Generada por el servidor al subir imagen.

status	string	✓ Sí	Estado. Enum: ACTIVE INACTIVE SOLD DECEASED. Default: ACTIVE
healthStatus	string	✓ Sí	Estado de salud. Enum: HEALTHY ALERT CRITICAL. Calculado automáticamente.
deviceId	string null	✗ No	ID del collar IoT asignado. Único en el sistema. Si null, animal sin sensor.
createdAt	string (ISO 8601)	✗ No	Timestamp de creación. Generado por el servidor.
updatedAt	string (ISO 8601)	✗ No	Timestamp de última actualización. Generado por el servidor.

5.2.2 Esquema: GPSPoint

Campo	Tipo	Req.	Descripción y validaciones
id	integer (bigint)	✗ No	ID autoincremental. Generado por el servidor.
animalId	string (cuid)	✓ Sí	ID del animal al que pertenece este punto.
lat	number	✓ Sí	Latitud en grados decimales. Min: -90, Max: 90. Precisión: 8 decimales.
lng	number	✓ Sí	Longitud en grados decimales. Min: -180, Max: 180. Precisión: 8 decimales.
alt	number null	✗ No	Altitud sobre el nivel del mar en metros.
accuracy	number null	✗ No	Precisión GPS en metros. Generalmente 3-15m para GPS civil.
speedKmh	number null	✗ No	Velocidad en km/h calculada desde el dispositivo.
recordedAt	string (ISO 8601)	✓ Sí	Timestamp del momento de la lectura GPS. Proviene del dispositivo IoT.

5.2.3 Esquema: Alert

Campo	Tipo	Req.	Descripción y validaciones
id	string (cuid)	✗ No	Generado por el servidor.
farmId	string (cuid)	✓ Sí	ID de la finca. Filtrado automáticamente por JWT.
animalId	string (cuid)	✓ Sí	ID del animal que originó la alerta.

geofenceId	string null	✗ No	ID de la geocerca involucrada. Solo en alertas de tipo GEOFENCE_EXIT.
ruleId	string	✓ Sí	Identificador de la regla disparada. Ej: HIGH_TEMP, GEOFENCE_EXIT, SENSOR_OFFLINE.
priority	string	✓ Sí	Enum: CRITICAL WARNING INFO.
status	string	✓ Sí	Enum: OPEN ACKNOWLEDGED ASSIGNED CLOSED. Default: OPEN
title	string	✓ Sí	Título descriptivo. Máximo 200 caracteres.
description	string	✓ Sí	Descripción detallada del evento. Máximo 1000 caracteres.
metadata	object null	✗ No	Datos adicionales en JSON libre. Ej: { tempC: 40.8, threshold: 39.5 }
assigneeId	string null	✗ No	ID del usuario asignado para resolver la alerta.
resolution	string null	✗ No	Nota de resolución al cerrar la alerta. Máximo 500 caracteres.
openedAt	string (ISO 8601)	✗ No	Timestamp de generación de la alerta.
closedAt	string null	✗ No	Timestamp de cierre. Null si la alerta sigue abierta.
responseMs	integer null	✗ No	Tiempo de respuesta en milisegundos (openedAt → closedAt).

5.3 Respuesta de Error Estándar

Todos los endpoints de la API devuelven errores en el mismo formato para facilitar el manejo uniforme en el cliente:

Campo	Tipo	Req.	Descripción
statusCode	integer	✓ Sí	Código de estado HTTP. Ej: 400, 401, 403, 404, 429, 500.
error	string	✓ Sí	Código de error interno. Ej: VALIDATION_ERROR, UNAUTHORIZED, NOT_FOUND.
message	string	✓ Sí	Mensaje legible por humanos. Siempre en español para el usuario final.
details	array null	✗ No	Array de errores de validación: [{ field, message }]. Solo en VALIDATION_ERROR.
requestId	string	✓ Sí	UUID v4 del request para correlación con logs del servidor.

timestamp	string	✓ Sí	Timestamp ISO 8601 del momento del error.
-----------	--------	------	---

5.4 Paginación Estándar

Todos los endpoints que retornan listas usan el mismo esquema de paginación:

Parámetro/Campo	Tipo	Descripción
?page	integer	Número de página. Default: 1. Min: 1.
?pageSize	integer	Registros por página. Default: 20. Min: 1, Max: 100.
?sortBy	string	Campo por el cual ordenar. Varía por endpoint. Default: createdAt.
?sortOrder	enum	Dirección de ordenamiento. Enum: asc desc. Default: desc.
data	T[]	Array de objetos del tipo correspondiente.
total	integer	Total de registros que coinciden con los filtros aplicados.
page	integer	Página actual retornada.
pageSize	integer	Tamaño de página efectivo.
totalPages	integer	Total de páginas disponibles. Calculado: ceil(total / pageSize).
hasNextPage	boolean	Indica si existe una página siguiente.

6. Interfaz MQTT — Dispositivos IoT

6.1 Convenciones de Topics

Todos los topics MQTT siguen la convención jerárquica: {dominio}/{finca}/{entidad}/{id}/{canal}. El separador es la barra /. Los componentes con llaves {} son variables.

Topic	QoS	Descripción y dirección del flujo
smartcow/farms/{farmId}/animals/{animalId}/telemetry	1	Telemetría principal. Dirección: Dispositivo → Broker → API Server. Publicado cada 10 segundos por el collar.
smartcow/devices/{deviceId}/status	1	Estado del dispositivo (online/offline/battery). Dirección: Dispositivo → API Server.
smartcow/devices/{deviceId}/config	2	Configuración remota del dispositivo (intervalo de envío, umbrales). Dirección: API Server → Dispositivo. QoS 2 para garantizar exactamente-una-vez.
smartcow/devices/{deviceId}/ota	2	Actualización de firmware OTA (Over-the-Air). Dirección: API Server → Dispositivo. Contiene URL del firmware en S3 y checksum SHA-256.
smartcow/farms/{farmId}/emergency	2	Canal de emergencia. El dispositivo puede publicar una alerta de emergencia (botón físico en el collar). QoS 2 para entrega garantizada.

6.2 Esquema del Payload de Telemetría

El payload publicado en el topic de telemetría es JSON. El tamaño máximo del payload es 512 bytes para optimizar el uso de ancho de banda en redes LoRa.

Campo	Tipo JSON	Req.	Descripción y rango válido
d	string	✓ Sí	deviceId: UUID v4 del collar. Configurado en flash del ESP32 en fabricación.
ts	number	✓ Sí	Timestamp Unix en milisegundos (UTC). Obtenido del módulo GPS o NTP.
la	number	✓ Sí	Latitud. Decimal degrees WGS84. -90 a 90. Precisión mínima: 6 decimales.
lo	number	✓ Sí	Longitud. Decimal degrees WGS84. -180 a 180. Precisión mínima: 6 decimales.
al	number null	✗ No	Altitud en metros sobre el nivel del mar. Null si GPS no la provee.

ac	number	✓ Sí	Precisión GPS en metros. 0 a 50. Valor > 30 indica señal GPS deficiente.
sp	number	✓ Sí	Velocidad en km/h. 0 a 200. Calculada por el módulo GPS.
tp	number	✓ Sí	Temperatura corporal en °C × 10 (entero). Ej: 384 = 38.4°C. DS18B20.
hr	number	✓ Sí	Ritmo cardíaco en bpm. 0 a 250. Sensor óptico.
ax	number	✓ Sí	Índice de actividad 0-100. Calculado del acelerómetro MPU-6050.
bt	number	✓ Sí	Nivel de batería 0-100%. Calculado del voltaje de la celda LiPo.
fv	string	✓ Sí	Versión de firmware. Formato semver: '1.2.3'.

Optimización de payload: Los nombres de campos son abreviaciones de 2 letras para minimizar el tamaño del JSON en redes LoRa con ancho de banda muy limitado (250 bps - 50 kbps). El servidor los mapea a nombres completos al procesar.

6.3 Autenticación de Dispositivos

Elemento de Seguridad	Especificación
Protocolo de conexión	MQTTS (MQTT sobre TLS 1.3). Puerto: 8883. Sin soporte de MQTT sin TLS en producción.
Autenticación de cliente	Certificado de cliente X.509 firmado por la CA del sistema. El broker Mosquitto valida el certificado en el handshake TLS. El ClientID del MQTT debe coincidir con el CN del certificado.
Emisión de certificados	El administrador genera el certificado de un nuevo dispositivo desde el panel de administración (Admin Panel → Dispositivos IoT → Nuevo dispositivo). El sistema genera el par de claves RSA-2048 y firma con la CA interna.
Validez del certificado	TTL: 2 años. Renovación automática vía canal OTA 30 días antes del vencimiento. El certificado vencido no puede conectar al broker.
Revocación	El administrador puede revocar un certificado (collar robado o dañado). El servidor actualiza la CRL (Certificate Revocation List) del broker. El dispositivo revocado no puede reconectarse.

7. Interfaz WebSocket — Tiempo Real

7.1 Protocolo de Conexión

El cliente web establece una conexión WebSocket con el servidor usando la librería Socket.IO 4.x. Socket.IO proporciona reconexión automática, heartbeats y namespaces que simplifican la gestión de conexiones en tiempo real.

Parámetro	Valor / Descripción
URL de conexión	wss://api.smartcow.app
Namespace	/farms/{farmId}
Autenticación	JWT enviado en la opción auth.token de Socket.IO al momento de la conexión. El servidor valida el token antes de aceptar la conexión. Conexión rechazada con 401 si el token es inválido.
Heartbeat	pingInterval: 25,000ms. pingTimeout: 10,000ms. Si el cliente no responde al ping, se considera desconectado y se elimina de las rooms activas.
Reconexión automática	Socket.IO cliente: reconnectionAttempts: Infinity, reconnectionDelay: 1000ms, reconnectionDelayMax: 30,000ms (backoff exponencial). El frontend muestra un indicador de 'Reconectando...' al usuario.
Rooms	Al conectar, el servidor une al cliente a la room de su finca: socket.join('farm:\${farmId}'). Los eventos se emiten solo a la room correspondiente, garantizando aislamiento entre fincas.

7.2 Eventos del Servidor al Cliente (Server → Client)

Evento	Payload (JSON Schema)	Descripción y frecuencia
animal:position	{ animalId, lat, lng, ts, speed }	Nueva posición GPS. Emitido por cada animal activo cada ~10s. Solo se emite si la posición cambió más de 2 metros respecto a la anterior.
animal:vitals	{ animalId, tempC, heartBpm, activity, battery }	Actualización de signos vitales. Acompaña siempre a animal:position. El cliente actualiza los VitalCards en tiempo real.
animal:status	{ animalId, healthStatus, prevStatus }	Cambio de estado de salud del animal (ej: HEALTHY → ALERT). Emitido inmediatamente cuando el Alert Engine cambia el estado.
alert:new	{ alertId, priority, ruleId, animalId, title }	Nueva alerta generada. El frontend muestra notificación toast y actualiza

		el badge de la campana y el listado de alertas.
<code>alert:updated</code>	<code>{ alertId, status, updatedBy, updatedAt }</code>	Estado de una alerta cambió (reconocida, asignada, cerrada). Actualiza el AlertCenter en tiempo real para todos los usuarios de la finca.
<code>sensor:offline</code>	<code>{ animalId, deviceId, lastSeenAt }</code>	Sensor dejó de transmitir por más de 5 minutos. El marcador del animal cambia a gris en el mapa.
<code>sensor:online</code>	<code>{ animalId, deviceId, reconnectedAt }</code>	Sensor volvió a estar en línea. El marcador del animal vuelve a su color de estado de salud.
<code>geofence:event</code>	<code>{ animalId, geofenceId, type, coords, ts }</code>	Animal cruzó el límite de una geocerca. type: ENTER EXIT. Inmediato, no hay polling.

7.3 Eventos del Cliente al Servidor (Client → Server)

Evento	Payload	Descripción
<code>subscribe:animal</code>	<code>{ animalId }</code>	Suscribe al cliente a eventos específicos de un animal (para la vista de detalle). El servidor añade al socket a la room del animal.
<code>unsubscribe:animal</code>	<code>{ animalId }</code>	Cancela la suscripción al animal. Ejecutado al cerrar el panel de detalle.
<code>ping</code>	<code>{ ts }</code>	Ping manual del cliente para verificar latencia de la conexión. El servidor responde con un evento pong.

8. Interfaces de Servicios Externos

8.1 SendGrid — Notificaciones por Email

Elemento	Especificación
Endpoint	POST <code>https://api.sendgrid.com/v3/mail/send</code>
Autenticación	Header: Authorization: Bearer {SENDGRID_API_KEY}. La clave se almacena en el secret manager del cloud, nunca en código.
Templates	Se usan templates dinámicos de SendGrid por tipo de alerta: d-alert-critical (rojo), d-alert-warning (ámbar), d-sensor-offline, d-weekly-report. Cada template acepta <code>dynamic_template_data</code> con los datos específicos.
Remitente	From: <code>alertas@smartcow.app</code> (dominio verificado con SPF + DKIM). Reply-To: <code>soporte@smartcow.app</code> .
Manejo de errores	Si SendGrid devuelve 5xx, Bull MQ reintenta con backoff exponencial (3 reintentos, delays: 30s, 2min, 10min). Los bounce y unsubscribe se procesan via webhook de SendGrid.

8.2 Twilio — Notificaciones por SMS

Elemento	Especificación
Endpoint	POST <code>https://api.twilio.com/2010-04-01/Accounts/{SID}/Messages.json</code>
Autenticación	HTTP Basic Auth: TWILIO_ACCOUNT_SID como usuario, TWILIO_AUTH_TOKEN como contraseña.
Número remitente	Número de teléfono SmartCow registrado en Twilio (+57...). Capacitado para SMS en Colombia.
Limitación de envíos	Máximo 3 SMS por usuario por hora para alertas del mismo tipo (anti-flood). Implementado con contador en Redis con TTL de 1 hora.
Plantilla de mensaje	 SmartCow CRÍTICO: {descripción corta}. Finca: {nombre}. Animal: {nombre/código}. {timestamp}. Ver: <code>app.smartcow.app</code>

8.3 Almacenamiento de Archivos (S3 Compatible)

Tipo de archivo	Path en bucket	Descripción
Fotos de animales	<code>animals/{farmId}/{animalId}/{uuid}.jpg</code>	Subidas desde el panel de administración. Máx: 5MB. Procesadas con Sharp:

		redimensionadas a 800×600px, convertidas a WebP.
Reportes exportados	<code>reports/{farmId}/{year}/{month}/{uuid}.xlsx</code>	Generados por el Report Service. URL firmada con expiración de 1 hora. Enviada al usuario por email.
Firmware IoT	<code>firmware/{version}/smart-cow-collar.bin</code>	Binario de firmware para actualización OTA. Acceso público con URL firmada enviada al dispositivo.
Backups de BD	<code>backups/{date}/db-{timestamp}.sql.gz</code>	Generados diariamente. Encriptados con AES-256. Retención: 30 días.

9. Flujos de Navegación

9.1 Mapa de Navegación Principal

SmartCow Tracker usa un modelo de navegación en árbol de un nivel de profundidad desde el sidebar, complementado con paneles laterales deslizantes para contenido secundario (no hay páginas anidadas en el sentido clásico).

Ruta (URL)	Nombre de pantalla	Acceso mínimo	Panel lateral disponible
/dashboard	Dashboard	OPERATOR	Detalle de animal (click en chip).
/map	Mapa GPS en tiempo real	OPERATOR	Detalle de animal (click en marcador). Editor de geocerca (modo dibujo).
/cattle	Listado de ganado	OPERATOR	Detalle de animal (click en tarjeta).
/cattle/:id	Ficha completa del animal	OPERATOR	Historial de rutas. Panel de edición (ADMIN/VET).
/alerts	Centro de alertas	OPERATOR	Contexto de la alerta (click en ítem).
/analytics	Dashboard analítico	VET / ADMIN	Filtros avanzados.
/admin/users	Gestión de usuarios	ADMIN	Formulario de usuario.
/admin/geofences	Gestión de geocercas	ADMIN	Editor de polígono.
/admin/devices	Gestión de dispositivos IoT	ADMIN	Detalle del dispositivo.
/settings	Configuración de la finca	ADMIN	Ninguno.
/login	Inicio de sesión	Público	Ninguno.

9.2 Flujo Crítico: Respuesta a Alerta

El flujo más común de uso en producción es la respuesta a una alerta crítica. El sistema está diseñado para minimizar el número de clics necesarios para tomar acción:

- El usuario recibe una notificación push/toast en el navegador: 'Res #B-042 fuera de geocerca'.
- Click en la notificación → navega automáticamente a /map con el animal B-042 centrado y seleccionado.
- El panel lateral derecho se abre automáticamente con el detalle del animal y la alerta activa.

4. El usuario evalúa la situación en el mapa (posición del animal respecto a la geocerca).
5. Click en 'Reconocer alerta' → estado cambia a ACKNOWLEDGED. El botón cambia a 'Cerrar' + campo de resolución.
6. El usuario escribe la nota de resolución y cierra la alerta. El evento alert:updated se emite a todos los demás usuarios de la finca.

Objetivo UX: *Este flujo completo (desde notificación hasta cierre de alerta) debe completarse en menos de 60 segundos sin necesidad de abrir otra pantalla.*

— Fin del Documento IDD v1.0.0 —

SmartCow Tracker | Interface Design Document | Mayo 2026